

CowControl — Guía de integración con la Raspberry Pi

Cómo conectar el lector RFID para que cada tag leído llegue a la app en tiempo real

1. La idea en una frase

Cada vez que el lector RFID detecta un tag, la Raspberry Pi tiene que mandar ese dato directo a una tabla de Supabase (`datos_lectura`) con una simple petición HTTP POST. Desde ahí, la app CowControl lo muestra solo, en tiempo real, sin que nadie tenga que tocar el celular. La Raspberry nunca habla con la app — las dos hablan con Supabase, que es el único "servidor" de todo el sistema.

2. Qué credenciales conseguir antes de arrancar

Dato	Dónde conseguirlo
<code>SUPABASE_URL</code>	Ya está en el archivo <code>.env</code> de la carpeta del proyecto Flutter.
<code>SERVICE_ROLE_KEY</code>	Dashboard de Supabase → Settings → API → Project API keys → service_role . No es la misma que la <code>anon key</code> del <code>.env</code> — esa es para la app, esta es para el hardware.
<code>CAMPO_ID</code>	Correr <code>SELECT id, nombre_campo FROM datos_campo;</code> en el SQL Editor de Supabase, y copiar el <code>id</code> del campo correspondiente.

La `service_role key` es secreta. Tiene permiso total sobre toda la base de datos, de todos los productores, sin ninguna restricción — nunca va dentro de la app, nunca se sube a un repositorio público, y no se comparte fuera del equipo del proyecto.

El `CAMPO_ID` es una constante fija por Raspberry, no algo dinámico. No hay ningún emparejamiento automático entre una Raspberry y una cuenta de usuario — cada Raspberry física que se instale tiene que tener su propio `CAMPO_ID` cargado a mano una sola vez, la primera vez que se configura.

3. El pedido HTTP que hay que mandar (para adaptar código ya existente)

Si ya hay código armado del lado del hardware, no hace falta reescribirlo desde cero — alcanza con que, en el momento de leer un tag, haga exactamente este pedido:

```
POST {SUPABASE_URL}/rest/v1/datos_lectura

Headers:
```

```
apikey: {SERVICE_ROLE_KEY}
Authorization: Bearer {SERVICE_ROLE_KEY}
Content-Type: application/json
```

Body (JSON):

```
{
  "rfid_uid": "<el UID que leyó el lector, como texto>",
  "campo_id": "<el CAMPO_ID de esta Raspberry>",
  "fecha_hora": "<fecha y hora actual, formato ISO 8601 UTC>"
}
```

Una respuesta 200 o 201 significa que se guardó bien. Si el `rfid_uid` coincide con un animal ya cargado en ese campo, en la app va a aparecer marcado con un ✓ al instante — si no coincide con ninguno, la lectura igual queda guardada en la tabla, solo que la app todavía no la asocia a ningún animal (ver sección 5).

4. Script de referencia completo (Python)

Si se prefiere arrancar de este script en vez de adaptar uno propio, esto ya está probado y listo para copiar. Usa el módulo lector RC522 (13.56 MHz) vía la librería `mfr522`.

Instalar dependencias (una sola vez)

```
pip3 install requests RPi.GPIO mfr522
```

rfid_tick.py

```
import time
from datetime import datetime, timezone

import requests
import RPi.GPIO as GPIO
from mfr522 import SimpleMFRC522

# — Configuración — completar antes de correr —
SUPABASE_URL = "PEGAR_ACA_LA_SUPABASE_URL_DEL_.ENV"
SUPABASE_SERVICE_ROLE_KEY = "PEGAR_ACA_LA_SERVICE_ROLE_KEY_DEL_DASHBOARD"
CAMPO_ID = "PEGAR_ACA_EL_UUID_DEL_CAMPO"
TIEMPO_IGNORAR = 5 # segundos para ignorar relecturas del mismo tag
# —————

headers = {
    "apikey": SUPABASE_SERVICE_ROLE_KEY,
    "Authorization": f"Bearer {SUPABASE_SERVICE_ROLE_KEY}",
    "Content-Type": "application/json",
}

lector = SimpleMFRC522()
ultimas_lecturas = {}

def ya_fue_leida_recientemente(uid):
```

```

uid_str = str(uid)
ahora = time.time()
if uid_str in ultimas_lecturas and ahora - ultimas_lecturas[uid_str] <
TIEMPO_IGNORAR:
    return True
ultimas_lecturas[uid_str] = ahora
return False

def mandar_lectura(uid):
    url = f"{SUPABASE_URL}/rest/v1/datos_lectura"
    payload = {
        "rfid_uid": str(uid),
        "campo_id": CAMPO_ID,
        "fecha_hora": datetime.now(timezone.utc).isoformat(),
    }
    try:
        respuesta = requests.post(url, json=payload, headers=headers, timeout=5)
        if respuesta.status_code in (200, 201):
            print(f"OK - Lectura guardada: {uid}")
        else:
            print(f"ERROR de Supabase ({respuesta.status_code}): {respuesta.text}")
    except requests.exceptions.RequestException as e:
        print(f"ERROR de red: {e}")

def bucle_principal():
    print("Lista. Acerca un tag al lector...\n")
    try:
        while True:
            try:
                uid, _texto = lector.read()
            except Exception as e:
                print(f"Error leyendo tag: {e}")
                time.sleep(1)
                continue

            if not ya_fue_leida_recientemente(uid):
                print(f"Tag leído: {uid}")
                mandar_lectura(uid)

            time.sleep(0.5)
    except KeyboardInterrupt:
        print("\nDetenido manualmente")
    finally:
        GPIO.cleanup()

if __name__ == "__main__":
    bucle_principal()

```

5. Qué pasa con un tag que no está cargado como animal todavía

La lectura se guarda igual en `datos_lectura` (nunca se descarta), pero como todavía no existe la función de "animales no identificados" del lado de la app (está diseñada pero no implementada), esa lectura por ahora no se ve reflejada como un animal nuevo en la Planilla. Para probar el circuito de punta a punta, lo más simple es usar el `rfid_uid` de un animal que ya esté cargado en la base (o

cargar uno de prueba a mano con una consulta `INSERT` en el SQL Editor).

6. Cómo confirmar que funciona

1. Abrir la app CowControl, loguearse con la cuenta dueña del campo configurado en `CAMPO_ID`.
2. Entrar a la Planilla o a "Iniciar conteo" (Conteo en vivo).
3. Acercar un tag físico al lector.
4. Sin tocar nada del celular, el animal correspondiente debería marcarse solo con un ✓ verde en la app, apenas unos segundos después de la lectura.

Cosas para no tocar sin avisar antes: no crear un proyecto de Supabase aparte (todo el equipo comparte el mismo, con la misma URL); no reemplazar las tablas existentes (`datos_campo`, `datos_animales`, `datos_lectura`, `datos_vacuna`) por otras nuevas — si hace falta un cambio de estructura, avisar antes de aplicarlo a la base de datos compartida.

7. Dudas o algo no cierra

Si algo de esto no coincide con el código que ya está armado del lado del hardware, no hay drama — la idea de esta guía es dar el "contrato" exacto que espera Supabase (URL, credenciales, formato del pedido), no reemplazar el trabajo ya hecho. Avisar por el grupo del equipo si algo no cierra o hace falta ajustar el nombre de algún campo.